

IMPLEMENTAZIONE SCOPING:

Statico $\Rightarrow$ Catena Statica

Raccolgo INFO a tempo di Compilazione (dal codice), riesco poi anche ad Evitare la RICERCA.

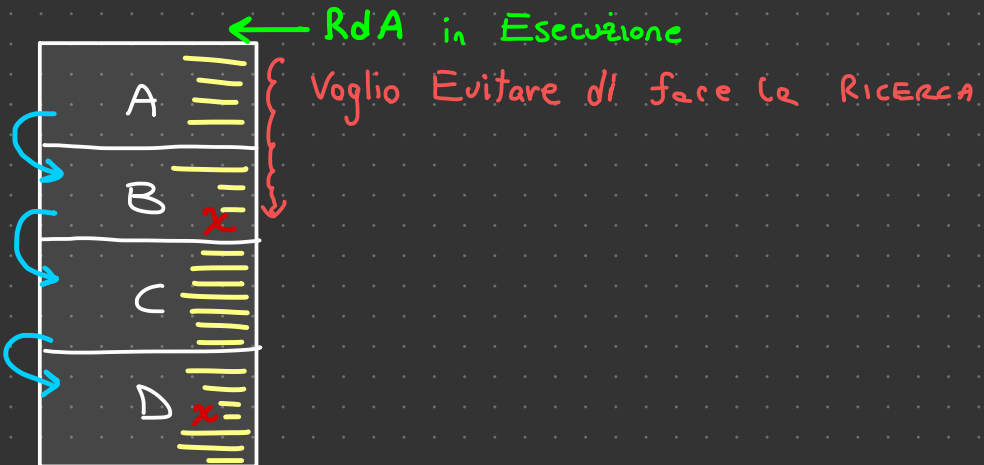
STRUTTURA ANNIDAMENTO
DELLE DEFINIZIONI

Sapendo Quanto devo Risolvere

Scoping Dinamico $\Rightarrow$ Per Ridurre Tempo Aumento Memoria
con Strutture dati Esterne (DA TENERE AGGIORNATA)

CRT $\Rightarrow$ Tabella Centrale dei Riferimenti, garantisce/Mantiene
aggiornata l'associazione Tra ID $\leftrightarrow$ AMBIENTE.

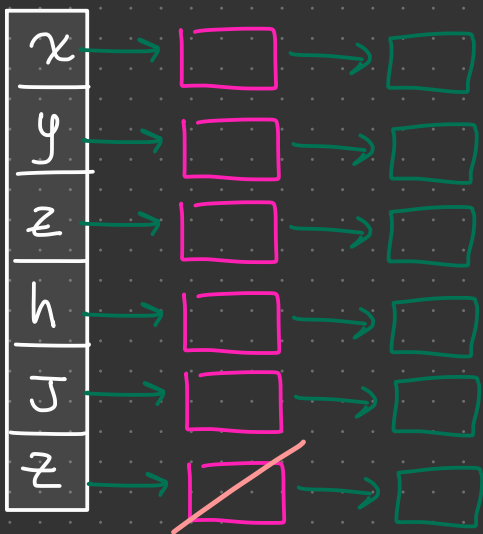
Costantemente devo Avere Accessibile l'ultimo Riferimento



Creo una Struttura Dati che Sfrutta lo SHALLOW ACCESS

Prendo un ARRAY DI ELEMENTI INDICIZZATI sul Nome delle Variabili.

Ogni ENTRY punta ad una Lista Linkata di Elementi, che ognuno Rappresenta/describe un AMBIENTE in cui la Variabile è definita



Sono TUTTI ambienti ATTIVI per cui l'Ambiente in TESTA è l'ultimo ARRIVATO (per EVITARE RICERCHE TROPPO LUNGHE) che è Quello di RIFERIMENTO per lo SCOPING DINAMICO

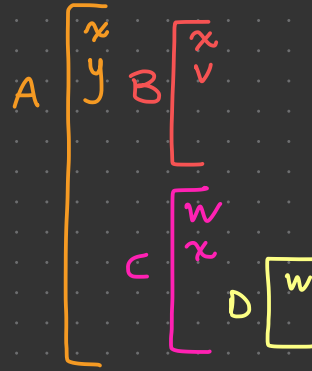
Così NON devo fare SCANSIONI $\Rightarrow$ Tempo di Accesso

Costante Garantito (ALLE INFO CHE MI PERMETTONO DI RECUPERARE LA VARIABILE)

Elimino la procedura ed Imposto Come ultimo ambiente il NEXT della PROCEDURA.

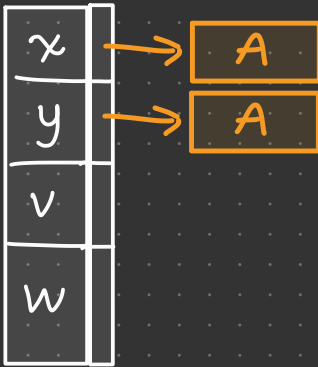
ESEMPIO:

$A \rightarrow B \rightarrow C \rightarrow D$

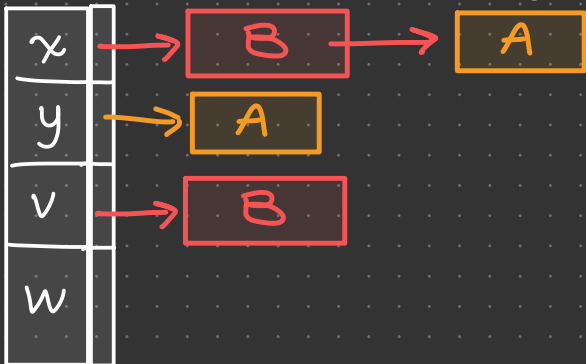


CRT:

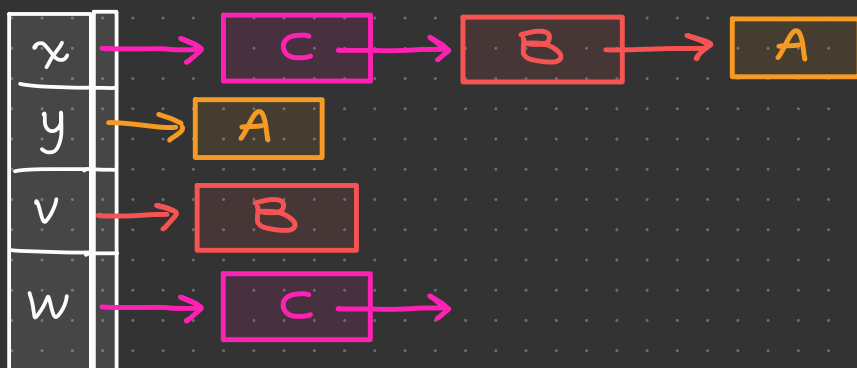
Quando Eseguo A, definisco x e y



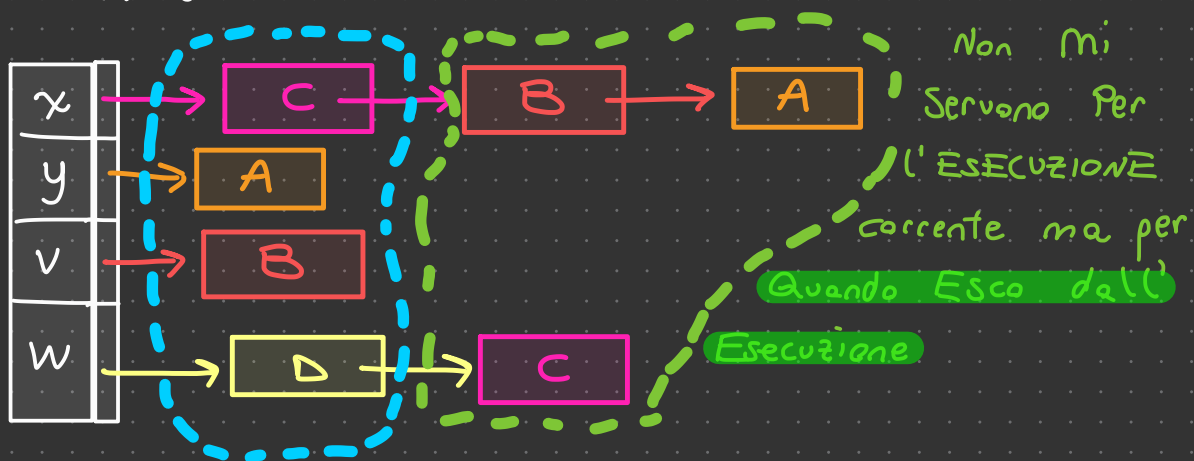
Passo Ad Eseguire B:



Poi Passo a C:



Ed in fine a D:



Utili Per l'ESECUZIONE CORRENTE

Mantenere ha dei **COSTI**, **AGGIORNAMENTO** ed **ELIMINAZIONE** ma **CONVIENE** per via dell' **ACCESSO DIRETTO** (dipende sempre poi dal programma)

Se Referenzio Molte Volte Sarà Sempre Vantaggioso

ESEMPIO:

```
{int b;
```

```
Void A () {
```

```
    int x=0, y=0, z=5;
```

```
    void B () {
```

```
        int x=3, w=3;
```

```
        x = y+z;
```

```
    }
```

```
Void C () {
```

```
    int y=1, v=1;
```

```
    void D () {
```

```
        int z=2, v=2;
```

```
        B();
```

```
        v = x+y;
```

```
    D();
```

```
    x = z+y;
```

```
}
```

```
    C(7);
```

```
}
```

```
    A();
```

```
}
```

B

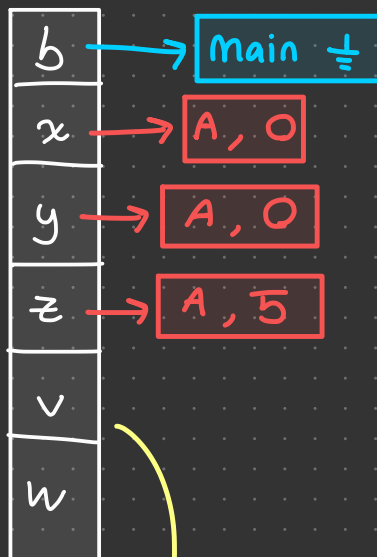
A

Main

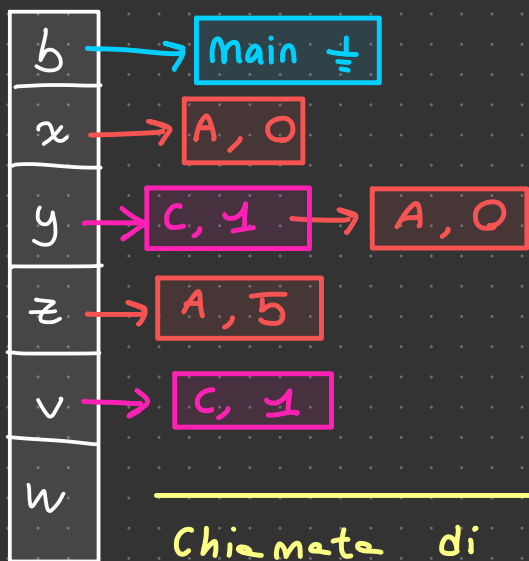
C



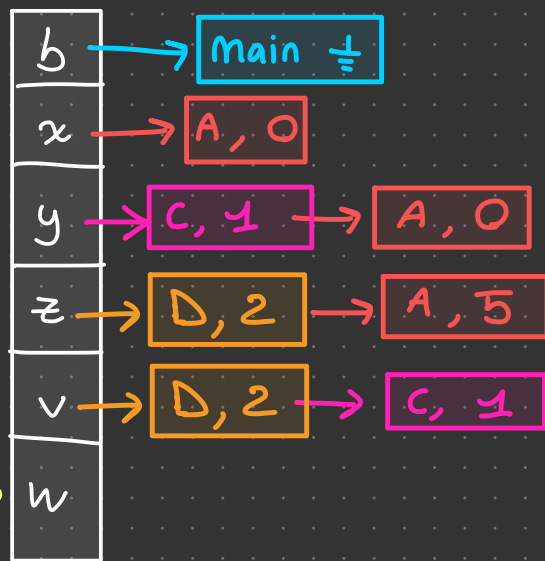
Chiamata di A



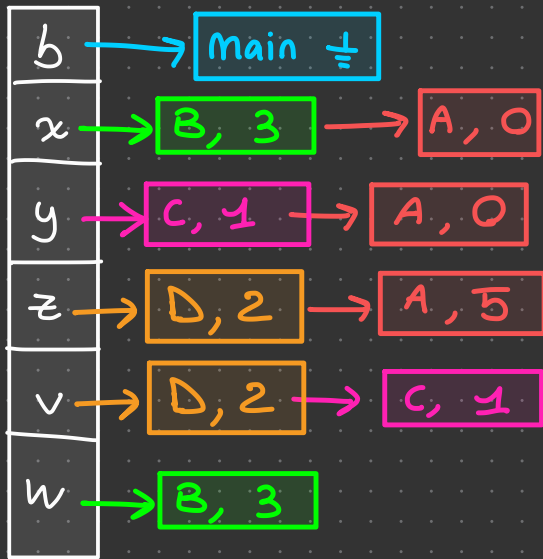
Chiamata C(z)



Chiamata di D()



Chiamata B() da D:



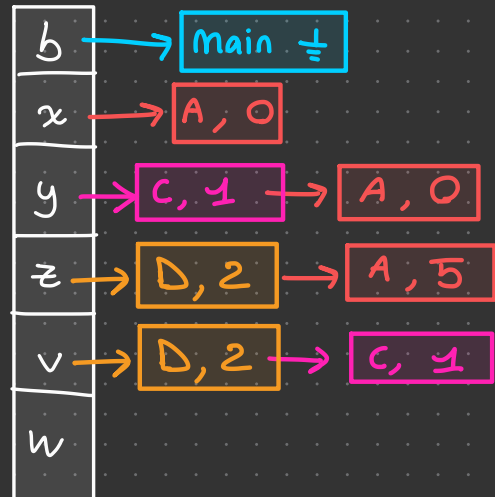
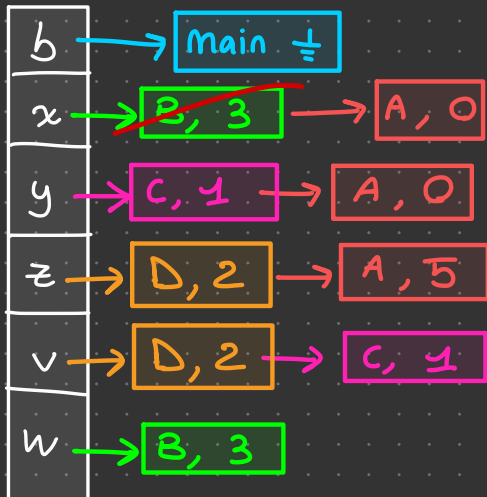
Prendo da C $\rightarrow 1$

Dopo l'EXEC di $x = y + z$

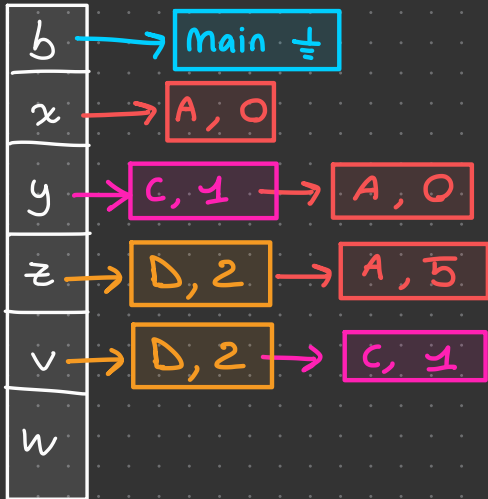
Prendo da D $\rightarrow 2$

che mi porta a Riscrivere 3 (NON C'È NESSUNA MODIFICA)

Ora Termina B quindi:

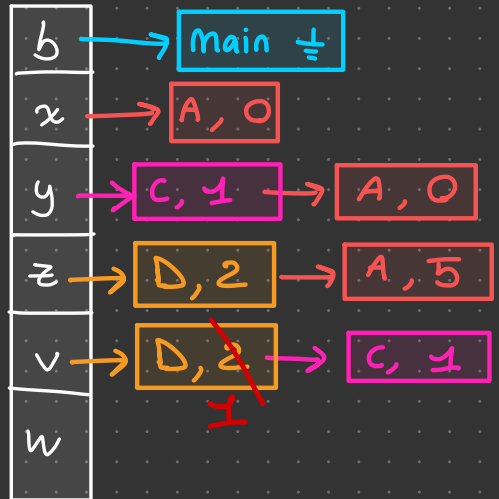


Torno in D:

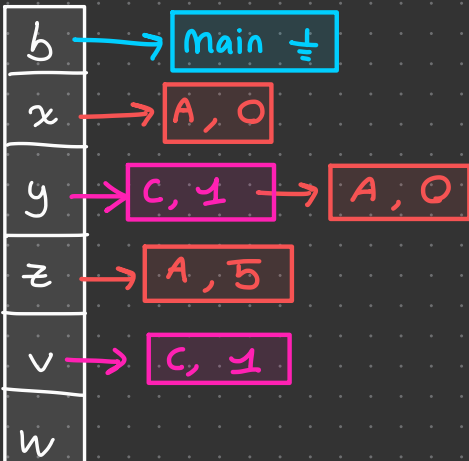


$C \rightarrow 1$
 $v = y + x$
 $A \rightarrow 0$

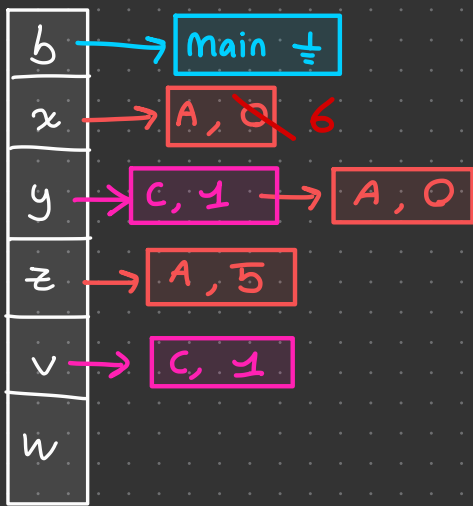
E quindi:



Termina D e Siamo in C

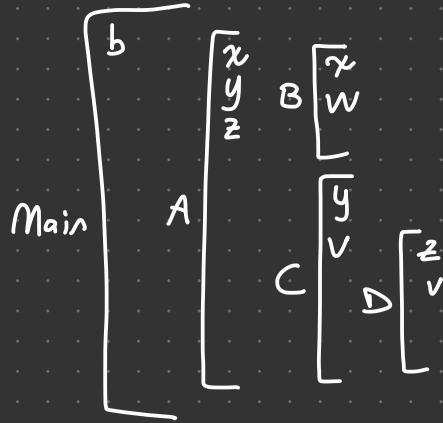


$A \rightarrow 5$
 $x = z + y \rightarrow C \rightarrow 1$



IL Resto delle chiamate distrugge le liste fino a svuotarle
Completamente (CRT Vuota)

PERCORSO SULLO STACK (scoping Statico):

$$S_d(\text{main}) = 0$$
$$\# \quad S_d(A) = 1$$
$$\# S_d(B) = S_d(C) = 2$$
$$\# \quad \text{Sd}(D) = 3$$


ESECUZIONE MAIN:



Chiamate di A

A	
x	0
y	0
z	5

Chiamata di $C(7)$

	C
a	7
y	1
v	1

~~→ Perm. formale~~

$$K = Sd(A) - Sd(C) + 1$$

$$\mathcal{L}_s(c) = A$$

(me lo aspetto, C è
definito dentro A)

Chiamo D()

D	
z	2
v	2

$$K_D = Sd(C) - Sd(D) + 1 = 0$$

$$Sd(D) = C$$

Chiamo B()

B	
x	3
w	3

$$K_B = Sd(D) - Sd(B) + 1$$

$$= 2$$

$$Ls(B) = Ls(Ls(D))$$

$$Ls(B) = A$$

Eseguo in B: $x = y + z$

(x però è locale, calcolo n per le
Altre 2 Variabili)

$$N_y: Sd(B) - Sd(A) + 1 \Rightarrow 1$$

$$N_z: Sd(B) - Sd(A) + 1 \Rightarrow 1$$

Per Trovare y e z Risolgo di 1 e mi Trovo in A

$$x = 0 + 5 \Rightarrow x = 5$$

Vado a Modificare

B	
x	5
w	3

Termino B e si Elimina il Suo RdA e la Situazione è Questa:

main	
b	1

A	
x	0
y	0
z	5

C	
a	7
y	1
v	1

D	
z	2
v	2

1

Devo Completare l'ESECUZIONE di D ed Eseguo:

locale $V = x + y$

$$N_y = Sd(D) - Sd(C) + 1 = 1$$

$$N_x = Sd(D) - Sd(A) + 1 = 2$$

$$V = 0 + 1 = 1$$

Sovrascrivo

Termino D ed Elimino RdA

ESE600 C:

main	
b	1

A	
x	6
y	0
z	5

C	
a	7
y	1
v	1

$x = z + y$
Riferito ad A

locale
 $y \rightarrow 1$

$$N_z = \text{Sd}(C) - \text{Sd}(A) + 1$$
$$= 1$$

$x = 5 + 1$ e vado a SOVRISCRIVERE

E via via si chiuderanno gli RDA fino al Main per poi svuotarsi completamente

RECAP BREVE:

Nei linguaggi, per usare le procedure Abbiamo Bisogno

① Dichiarazione $\Rightarrow$ Interfaccia $\Rightarrow$ Nome, Parametri, Codice/Corpo

② Comando per la Chiamata $\Rightarrow$ Eseguire Codice

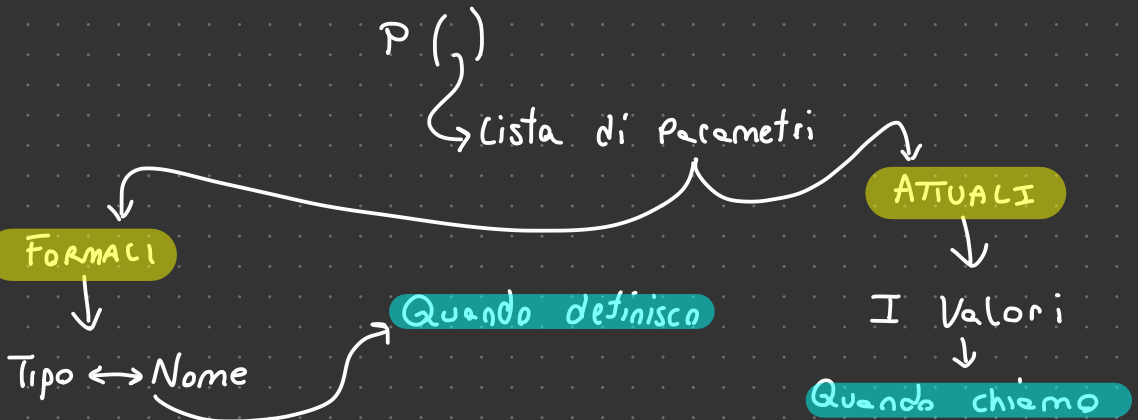
Dec ... $D \rightarrow \dots \mid \text{proc } P \text{ } () \text{ } C$ **Comandi**

$\swarrow$ Nome $\searrow$ Senza Param per ora

Com ... $C \rightarrow \dots \mid P ()$

$\swarrow$ La chiamata

Con Aggiunta dei parametri Avrà:



PARAMETRI DELLA PROCEDURA:

Permettono di far corrispondere ad un UNICO CODICE più ESECUZIONI.

C'è una COMUNICAZIONE Tra chiamante e Chiamato

DICHIARAZIONE: $\text{int } f(\text{int } n) \{ \text{return } n+1 \}$

↳ Valore Ritorno

↳ Nome

↳ Parametro formale che Quando uso Procedura avrà un Significato

Uso/CHIAMATA: $x = f(3);$ oppure $x = f(3+x)$

↳ Parametro Attuale

↳ r-Value

Direzione Comunicazione:

IN MODE $\Rightarrow$ dal Chiamante Verso Chiamato (Procedura),
Il Param. formale è Ambiente locale e riceve
il Valore dell'Attuale

OUT-MODE $\Rightarrow$ dal Chiamato al Chiamante, Passaggio di
Parametro per RISULTATO (poco USATI).

Out-Mode più comune è il RETURN.

/// In-Out $\Rightarrow$ Bidirezionale, ho Entrambi i TIPI, posso Separare i 2 Canali oppure no

MODELLO CONCETTUALE DI PASSAGGIO:

/// Sposto fisicamente il Valore su Nuova locazione che è il Passaggio per nome

/// Per Riferimento, Trasferisco il link di Accesso al Valore